

DATA-260 Homework 4
Rental Housing Listings Application

Student: Xuanhua Li
SID4: 2974
Domain: Rental Housing Listings
Submission Tag: hw4

GitHub Repository:
<https://github.com/carrovo/data260-2974.git>

Implementation Commit Hash:
c1978c6928a6719e1e503b5e945b57673a8656a7

Hardware: MacBook Air (Mac16,13), Apple M4, 10-core CPU
(4 performance and 6 efficiency cores), 16 GB memory

Local LLM: qwen3:8b through Ollama
Embedding Model: sentence-transformers/all-MiniLM-L6-v2
Tagged Commit Hash: [ADD AFTER FINAL COMMIT]
Assigned Domain: Rental housing listings

Value	Result	Calculation or Meaning
SID4	2974	Last four digits of student ID
PORT_BASE	8274	$8000 + (2974 \bmod 900)$
PREFIX	s2974	"s" + SID4
SEED	2974	Deterministic data seed
VERIFY_SEED	262974	$260000 + 2974$
DOMAIN_ID	6	$2974 \bmod 8$

Part 1 - React Client

This homework builds on HW3. Your existing FastAPI backend becomes the backend for the React client below, and your domain data moves to MySQL. Keep extending your existing service - don't start a new one.

Create a domain application where users can add, update, and delete records for your assigned domain entity.

Add a Login page/component (Login.jsx) that allows a user to log in using email + Password.

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

import { loginUser } from "../api";

export default function Login({ onLogin }) {
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  // Submit the login form.
  async function handleSubmit(event) {
    event.preventDefault();

    setError("");
    setLoading(true);

    try {
      // The browser stores the HTTP-only session cookie.
      const loginResult = await loginUser({
        email,
        password,
      });

      // Wait until App.jsx confirms the current user.
      await onLogin(loginResult);

      // Navigate only after the session is ready.
      navigate("/");
    }
  }
}
```

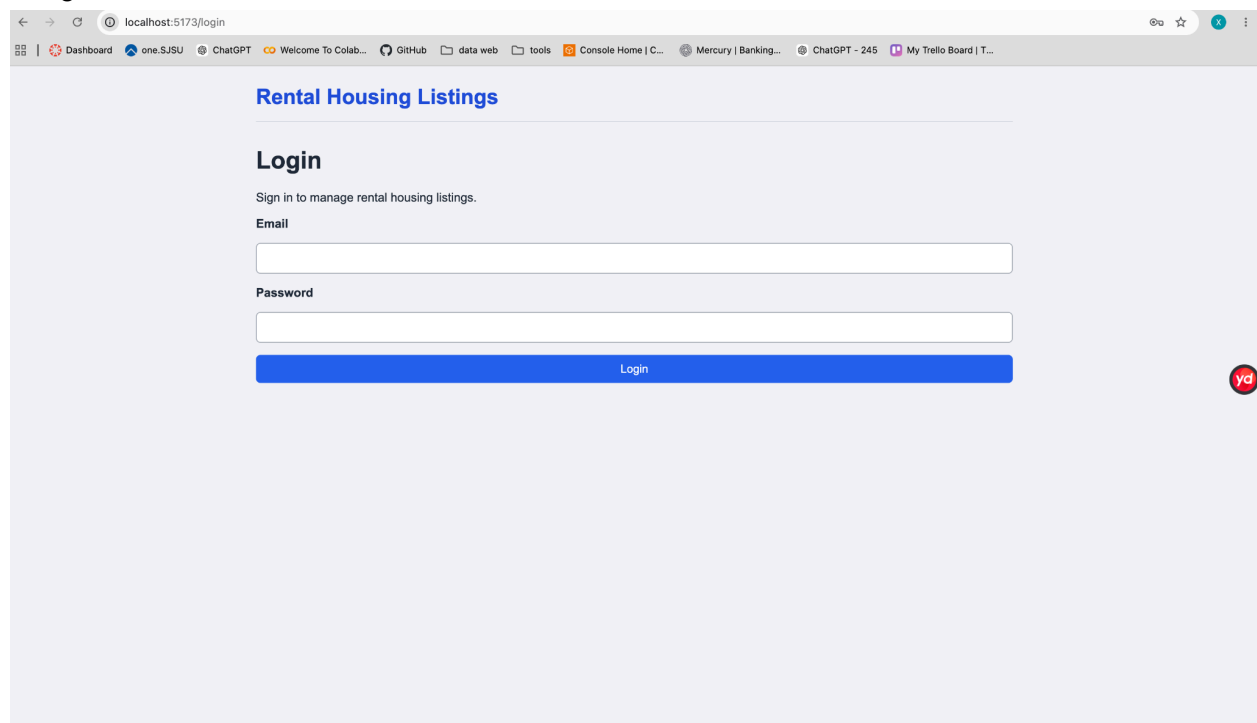
```

<Routes>
  <Route
    path="/login"
    element={
      user ? (
        <Navigate to="/" replace />
      ) : (
        <Login onLogin={handleLogin} />
      )
    }
  />

  <Route
    path="/"
    element={
      <RequireAuth user={user} loading={authLoading}>
        <Home
          user={user}
          listings={listings}
          loading={listingsLoading}
        />
      </RequireAuth>
    }
  />

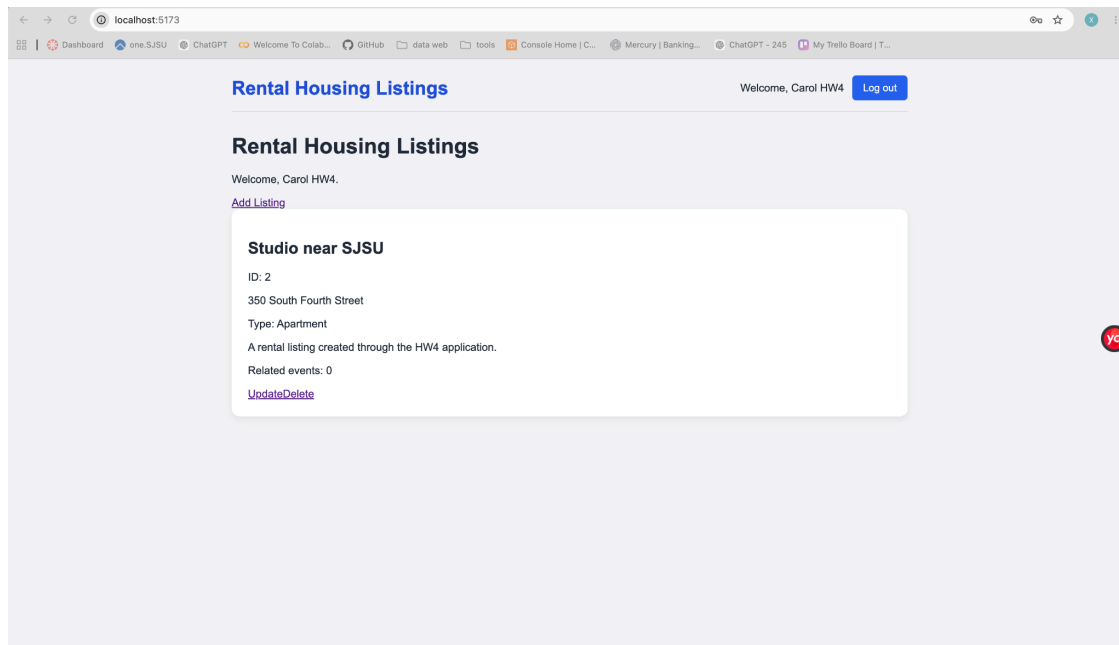
```

- After successful login, the backend will set an HTTP-only cookie containing a server-side session token.
- Only logged-in users should be allowed to view the record list and perform CRUD operations.
- If the user is not logged in: show a message "Login required", and/or disable/hide navigation buttons like "Add Record".



The screenshot shows a web browser window with the address bar displaying 'localhost:5173/login'. The browser's tab bar includes several open tabs: 'Dashboard', 'one.SJSU', 'ChatGPT', 'Welcome To Colab...', 'GitHub', 'data web', 'tools', 'Console Home | C...', 'Mercury | Banking...', 'ChatGPT - 245', and 'My Trello Board | T...'. The main content of the page is titled 'Rental Housing Listings' in a blue header. Below the header, the section is titled 'Login'. A sub-header reads 'Sign in to manage rental housing listings.' There are two input fields: one labeled 'Email' and another labeled 'Password'. Below these fields is a blue button labeled 'Login'. A small red circular logo with the letters 'yd' is visible in the bottom right corner of the page.

The login page displayed after logout.



The authenticated rental-listing home page.

I. Home Page

Write the necessary code to create the Home.jsx component which will display all the records. The Home component should be rendered when the user is on the root route /.

```
frontend > src > pages > Home.jsx > ...
1  import { Link } from "react-router-dom";
2
3  export default function Home({ user, listings, loading }) {
4    return (
5      <section>
6        <h2>Rental Housing Listings</h2>
7
8        <p>
9          Welcome, {user?.name || user?.email}.
10        </p>
11
12        <Link className="button-link" to="/create">
13          Add Listing
14        </Link>
15
16        {loading && <p>Loading listings...</p>}
17
18        {!loading && listings.length === 0 && (
19          <p className="empty-message">
20            No rental listings found.
21          </p>
22        )}
23
24        {!loading &&
25          listings.map((listing) => (
26            <article className="listing-card" key={listing.id}>
27              <h3>{listing.listingTitle}</h3>
28
29              <p>ID: {listing.id}</p>
30              <p>{listing.address}</p>
31              <p>Type: {listing.propertyType}</p>
32              <p>{listing.description}</p>
33              <p>
```

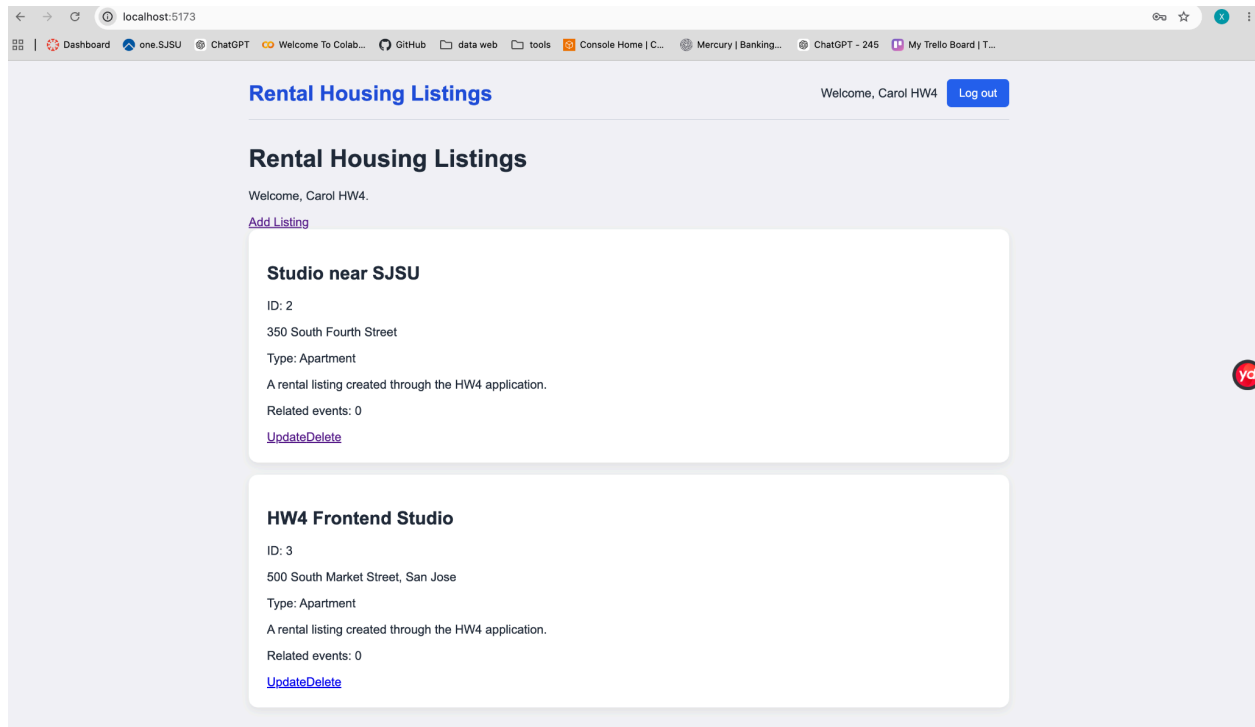
II. Add a New Record

Write the necessary code to create the `CreateRecord.jsx` component. The component should be rendered when the user is on the `/create` route.

The component should accept props to add the new record and have the following:

- An input field for entering the entity's primary field.
- An input field for entering the entity's secondary field.
- A submit button labeled appropriately for your domain.
- When the user clicks the submit button:
 - A new record should be added to the list with an auto-incremented ID.
 - The user should be redirected to the home page to see the updated list.

```
frontend > src > pages > CreateRecord.jsx > CreateRecord > handleSubmit
1  import { useState } from "react";
2  import { useNavigate } from "react-router-dom";
3
4  import { createListing } from "../api";
5
6
7  export default function CreateRecord({ onCreate }) {
8    const navigate = useNavigate();
9
10   const [listingTitle, setListingTitle] = useState("");
11   const [address, setAddress] = useState("");
12   const [error, setError] = useState("");
13   const [loading, setLoading] = useState(false);
14
15
16   async function handleSubmit(event) {
17     event.preventDefault();
18
19     setError("");
20     setLoading(true);
21
22     try {
23       // The backend supplies default values for the remaining fields.
24       const createdListing = await createListing({
25         listingTitle,
26         address,
27       });
28
29       onCreate(createdListing);
30       navigate("/");
31     } catch (requestError) {
32       const detail = requestError.response?.data?.detail;
33       setError(detail || "Unable to create the listing.");
34     }
35   }
36 }
```



Creating a new rental listing through the React client.

III. Update Record

Write the necessary code to create the `UpdateRecord.jsx` component. The component should be rendered when the user is on the `/update` route.

The component should accept props to update the record and have the following:

- An input field for the primary field.
- An input field for the secondary field.
- A submit button labeled appropriately for your domain.

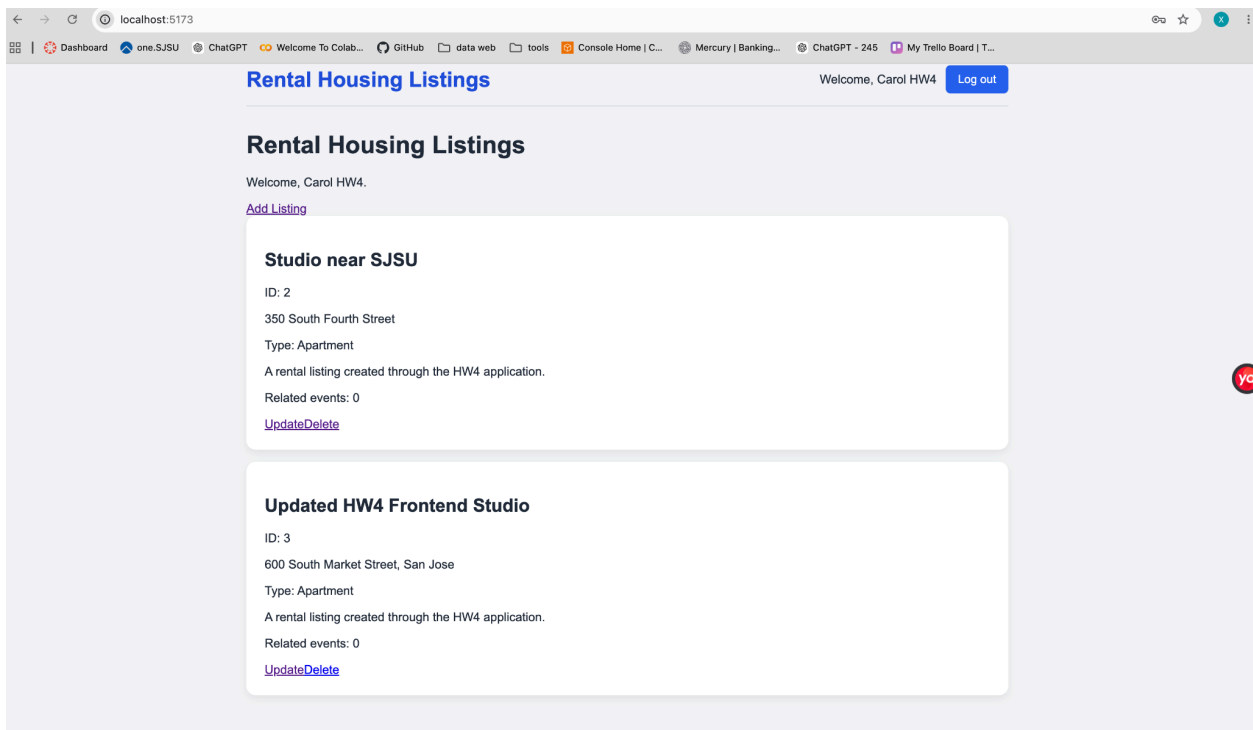
When the user clicks the submit button, the record should be updated through the backend API and persisted in MySQL and the user should be redirected to the home page showing the updated list.

```

frontend > src > pages > UpdateRecord.jsx > UpdateRecord > useEffect() callback > loadListing
1  import { useEffect, useState } from "react";
2  import { useNavigate, useParams } from "react-router-dom";
3  import { getListingById, updateListing } from "../api";
4
5  export default function UpdateRecord({ onUpdated }) {
6    const { id } = useParams();
7    const navigate = useNavigate();
8
9    const [form, setForm] = useState({
10      listingTitle: "",
11      address: "",
12    });
13    const [loading, setLoading] = useState(true);
14    const [saving, setSaving] = useState(false);
15    const [error, setError] = useState("");
16
17    // Load the existing listing before editing.
18    useEffect(() => {
19      let active = true;
20
21      async function loadListing() {
22        try {
23          const data = await getListingById(id);

```

UpdateRecord.jsx loads the selected listing by ID, places the current values in the form, and sends the updated title and address to the FastAPI PUT endpoint. The updated data is persisted in MySQL.



Updating a rental listing through the React client.

IV. Delete the Record

Write the necessary code to create the DeleteRecord.jsx component. The component should be rendered when the user is on the /delete route.

The component should accept props to delete a record and have the following:

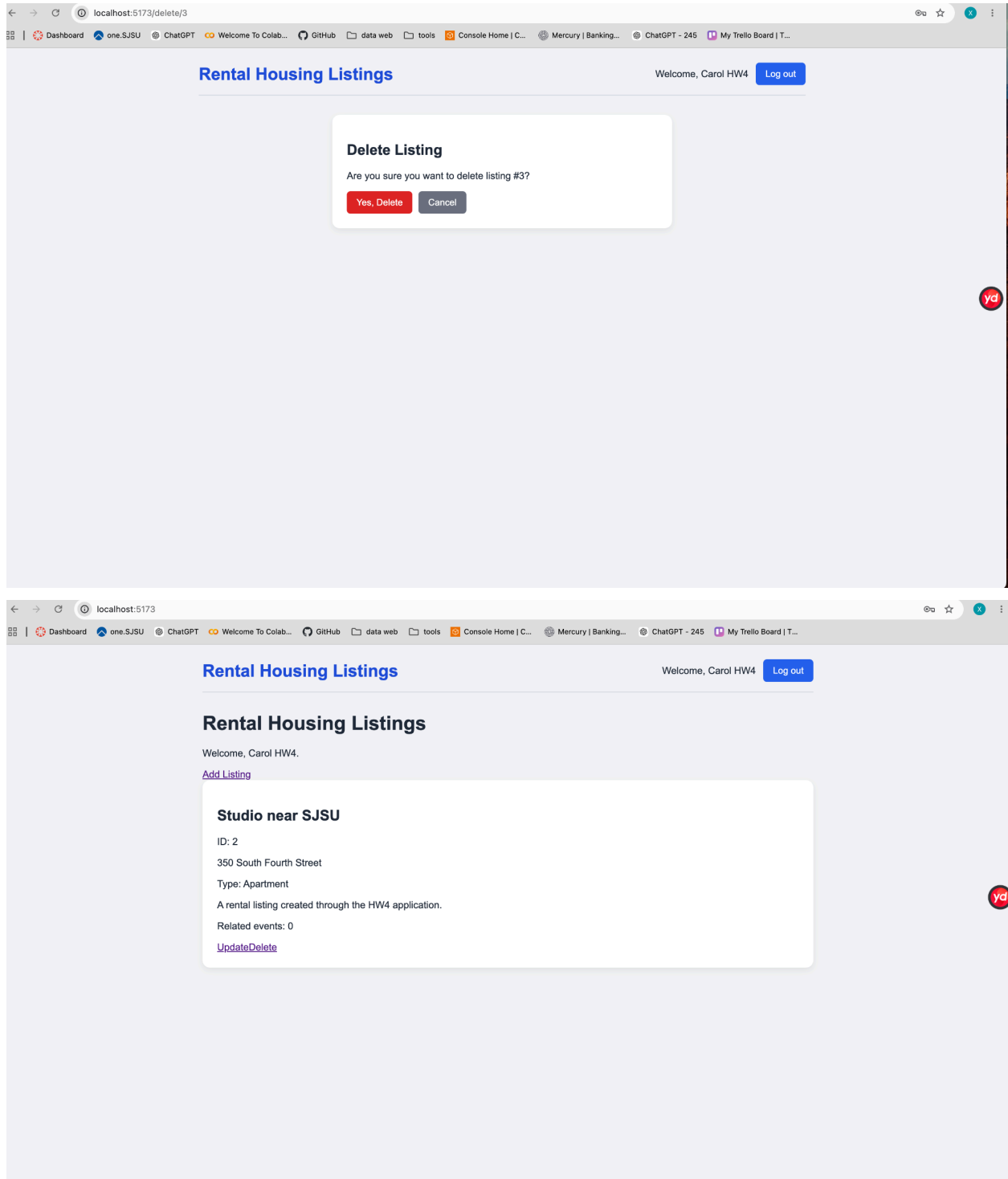
- A button labeled appropriately for your domain (e.g., "Delete").

When the user clicks the delete button, the record should be deleted through the backend API and removed from MySQL and the user should be redirected to the home page showing the updated list.

```
frontend > src > pages > DeleteRecord.jsx > DeleteRecord > handleDelete

1  import { useState } from "react";
2  import { useNavigate, useParams } from "react-router-dom";
3  import { deleteListing } from "../api";
4
5  export default function DeleteRecord({ onDelete }) {
6    const { id } = useParams();
7    const navigate = useNavigate();
8
9    const [deleting, setDeleting] = useState(false);
10   const [error, setError] = useState("");
11
12   // Delete the selected listing.
13   async function handleDelete() {
14     setDeleting(true);
15     setError("");
16
17     try {
18       await deleteListing(id);
19
20       onDelete(Number(id));
21       navigate("/");
22     } catch (err) {
23       setError(
24         err.response?.data?.detail || "Could not delete the listing."
25       );
26       setDeleting(false);
27     }
28   }
}
```

DeleteRecord.jsx displays a confirmation page before sending the DELETE request. After deletion, the listing is removed from MySQL and the user is redirected to the home page.



V. For all the above components, make sure to:

- Pass props for the Create, Update, and Delete components.
- Use hooks like `useState` and `useEffect` wherever necessary.
- Use React Router (`react-router-dom`) for routing

```
<Routes>

  <Route
    path="/login"
    element={
      user ? (
        <Navigate to="/" replace />
      ) : (
        <Login onLogin={handleLogin} />
      )
    }
  />

  <Route
    path="/"
    element={
      <RequireAuth user={user} loading={authLoading}>
        <Home
          user={user}
          listings={listings}
          loading={listingsLoading}
        />
      </RequireAuth>
    }
  />

  <Route
    path="/create"
    element={
      <RequireAuth user={user} loading={authLoading}>
        <CreateRecord onCreated={handleCreated} />
      </RequireAuth>
    }
  />

  <Route
    path="/update/:id"
    element={
      <RequireAuth user={user} loading={authLoading}>
        <UpdateRecord onUpdated={handleUpdated} />
      </RequireAuth>
    }
  />


```

```
<Route
  path="/delete/:id"
  element={
    <RequireAuth user={user} loading={authLoading}>
      <DeleteRecord onDelete={handleDeleted} />
    </RequireAuth>
  }
/>

<Route path="*" element={<Navigate to="/" replace />} />
</Routes>
```

React Router connects the login, home, create, update, and delete components. The components use `useState`, `useEffect`, `useNavigate`, and `useParams` where needed. Authentication state and callback functions are passed through props.

Part 2 - MySQL Persistence and Server-Side Sessions

In this part, you will apply the concepts learned in class to build a simple CRUD (Create, Read, Update, Delete) application using FastAPI and MySQL, using SQLAlchemy.

Database

Create MySQL database: <PREFIX>_rel

Table: your primary domain entity (auto-increment id, primary field, secondary field)

```
DATABASE_URL = os.getenv("DATABASE_URL")
# Required HW4 database connection variable name.
db_session_basede26 = create_engine(
    DATABASE_URL,
    pool_pre_ping=True,
)

# Create one SQLAlchemy session factory.
SessionLocal = sessionmaker(
    bind=db_session_basede26,
    autocommit=False,
    autoflush=False,
)
```

```
class RentalListing(Base):
    """Primary rental-listing entity for the HW4 domain application."""
    __tablename__ = "listings"

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        autoincrement=True,
    )
```

```
class User(Base):
    """Registered application user with a securely stored password hash."""

    __tablename__ = "users"

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        autoincrement=True,
    )
```

```

class SessionToken(Base):
    """Server-side login session referenced by an opaque browser cookie."""

    __tablename__ = "sessions"

    id: Mapped[str] = mapped_column(
        String(64),
        primary_key=True,
    )

    user_id: Mapped[int] = mapped_column(
        ForeignKey("users.id", ondelete="CASCADE"),
        nullable=False,
    )

```

```

class ListingEvent(Base):
    """Related test entity used to demonstrate the N+1 query problem."""

    __tablename__ = "listing_events"

    # Index event types for faster filtered event queries.
    __table_args__ = (
        Index(
            "idx_listing_events_event_type",
            "event_type",
        ),
    )

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        autoincrement=True,
    )

    listing_id: Mapped[int] = mapped_column(
        ForeignKey("listings.id", ondelete="CASCADE"),
        nullable=False,
    )

```

The application connects to the s2974_rel MySQL database through SQLAlchemy. The required connection variable is named db_session_basede26. The schema contains users, sessions, listings, and listing_events tables with foreign-key relationships.

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % mysql -h 127.0.0.1 -u s2974_app -p s2974_rel \
-E \
-e "SHOW CREATE TABLE sessions; SHOW CREATE TABLE listing_events;"
Enter password:
***** 1. row *****
Table: sessions
Create Table: CREATE TABLE `sessions` (
  `id` varchar(64) COLLATE utf8mb4_unicode_ci NOT NULL,
  `user_id` int NOT NULL,
  `created_at` datetime NOT NULL DEFAULT (now()),
  `expires_at` datetime NOT NULL,
  PRIMARY KEY (`id`),
  KEY `user_id` (`user_id`),
  CONSTRAINT `sessions_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
***** 1. row *****
Table: listing_events
Create Table: CREATE TABLE `listing_events` (
  `id` int NOT NULL AUTO_INCREMENT,
  `listing_id` int NOT NULL,
  `event_type` varchar(50) COLLATE utf8mb4_unicode_ci NOT NULL,
  `details` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `created_at` datetime NOT NULL DEFAULT (now()),
  PRIMARY KEY (`id`),
  KEY `listing_id` (`listing_id`),
  CONSTRAINT `listing_events_ibfk_1` FOREIGN KEY (`listing_id`) REFERENCES `listings` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB AUTO_INCREMENT=201 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
○ ((.venv) ) carol@Carols-MacBook-Air data260-2974 %

```

MySQL table fields, indexes, and foreign-key structure.

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % mysql -h 127.0.0.1 -u s2974_app -p s2974_rel \
-e "SHOW TABLES; DESCRIBE users; DESCRIBE sessions;"
Enter password:
+-----+
| Tables_in_s2974_rel |
+-----+
| listing_events      |
| listings            |
| sessions            |
| users               |
+-----+

+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+
| id         | int       | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100) | NO   |     | NULL    |
| email      | varchar(255) | NO   | UNI | NULL    |
| password_hash | varchar(255) | NO   |     | NULL    |
+-----+

+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+
| id         | varchar(64) | NO   | PRI | NULL    | |
| user_id    | int         | NO   | MUL | NULL    |
| created_at | datetime    | NO   |     | now()   | DEFAULT_GENERATED |
| expires_at | datetime    | NO   |     | NULL    |
+-----+
○ ((.venv) ) carol@Carols-MacBook-Air data260-2974 %

```

MySQL tables created for the HW4 application.

Also create tables for authentication/session management:

- users (id, name, email (unique), password_hash)
- sessions (id (session token), user_id, created_at, expires_at)

Name your database connection variable 'db_session_basede26' strictly

You are required to create a CRUD system for your domain entity that allows users to:

- Add a new record (POST)
- View all records (GET)
- View a record by ID (GET)
- Update record details (PUT)
- Delete a record (DELETE)

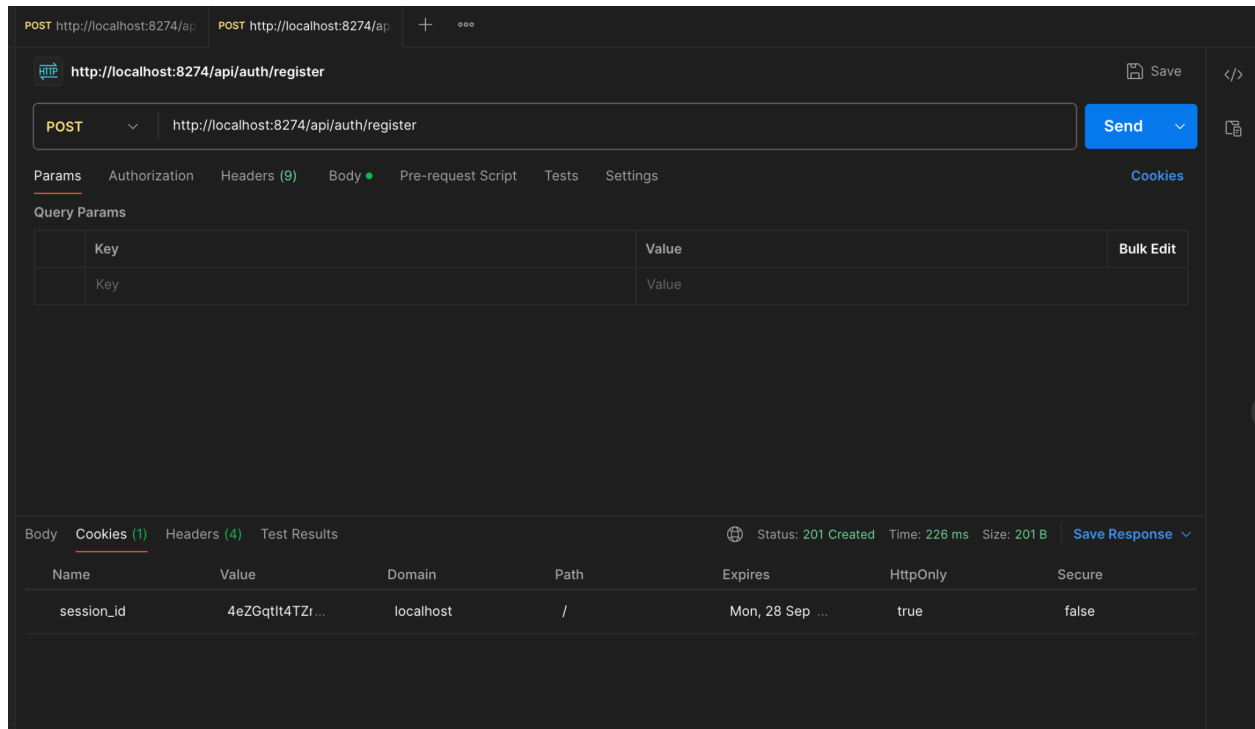
```
router = APIRouter(
    prefix="/api/auth",
    tags=["authentication"],
)

SESSION_COOKIE_NAME = "session_id"

# Local development uses HTTP, so Secure must be false.
# It should be enabled in production behind HTTPS.
COOKIE_SECURE = os.getenv(
    "COOKIE_SECURE",
    "false",
).lower() in {"true", "1", "yes"}
```

```
@router.post(
    "/register",
    response_model=UserOut,
    status_code=201,
)
def register(
    payload: UserCreate,
    db: Session = Depends(get_db),
) -> User:
    """Create a new user account with a bcrypt password hash."""
```

The registration endpoint validates the submitted email, hashes the password, and stores the new user in MySQL. Plain-text passwords are never stored in the database.

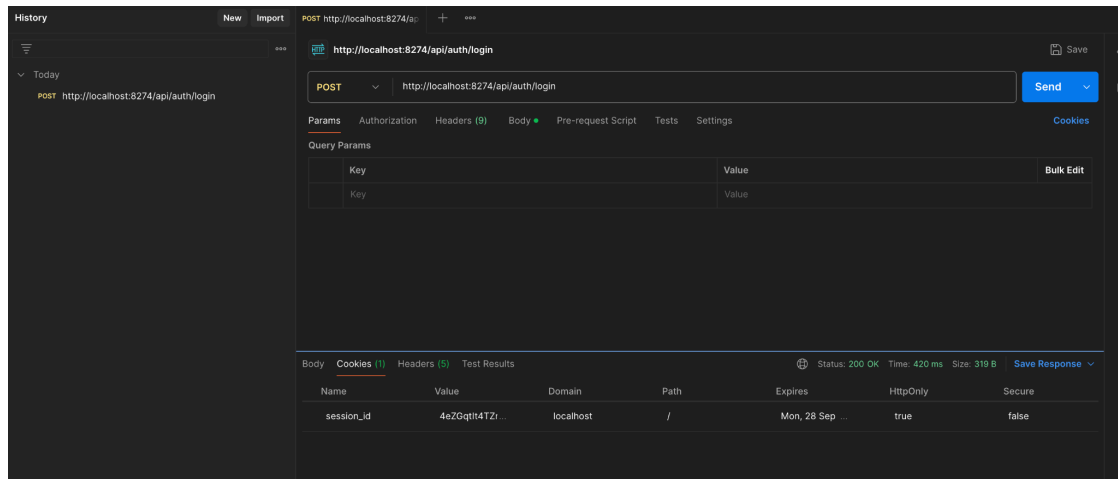


Successful user registration in Postman.

```
@router.post(
    "/login",
    response_model=LoginResponse,
)
def login(
    payload: UserLogin,
    response: Response,
    db: Session = Depends(get_db),
) -> LoginResponse:
    """Verify credentials and issue an HTTP-only session cookie."""

    user = get_user_by_email(db, str(payload.email))
```

After validating the email and password, the backend generates an opaque session token and stores it in the sessions table. The browser receives only the token in an HTTP-only cookie; the cookie does not contain user information.

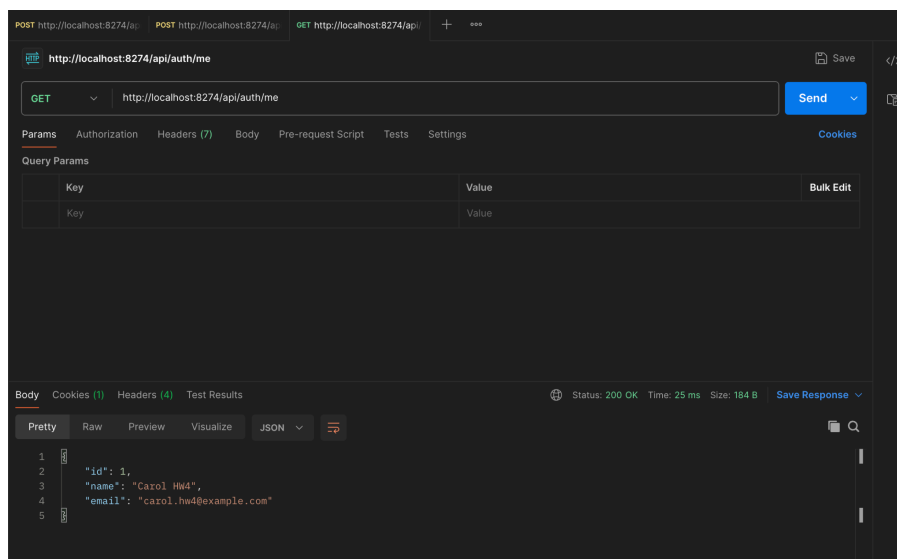


Successful login response and HTTP-only session cookie.

```
@router.get(
    "/me",
    response_model=UserOut,
)
def me(
    session: SessionToken = Depends(require_session),
    db: Session = Depends(get_db),
) -> User:
    """Return the user associated with the current session."""

    user = db.get(User, session.user_id)
```

The `/api/auth/me` endpoint reads the opaque cookie, validates the server-side session, and returns the authenticated user's public information.

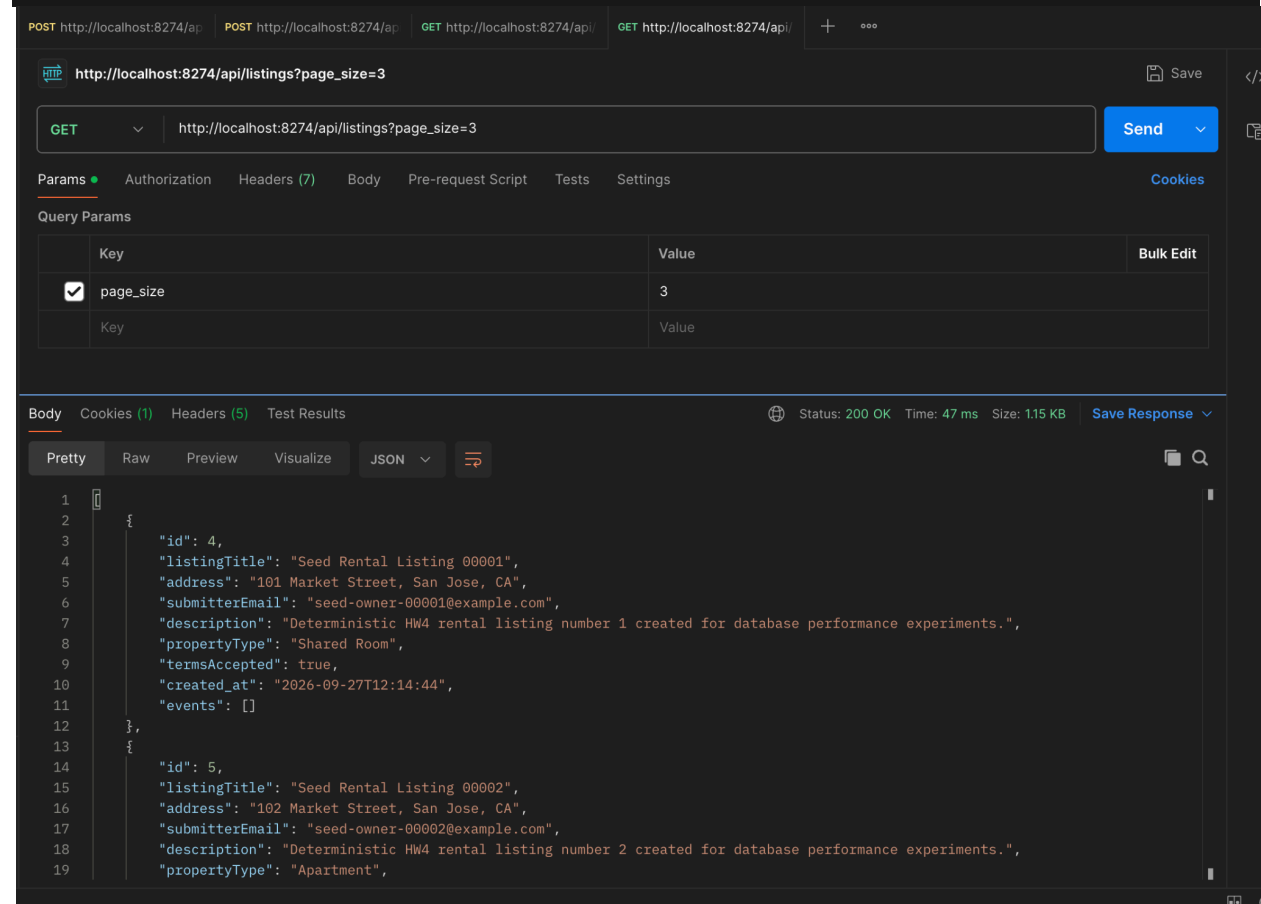


You are free to name your API endpoints appropriately.

The session token issued at login must be stored server-side in the sessions table and referenced only by an opaque token in the browser's HTTP-only cookie - the cookie itself should never store user data directly.

Additionally submit Postman API response screenshots for each operation. Also submit a screenshot of your database and your project folder structure.

```
def list_listings(
    response: Response,
    page_size: int = Query(
        default=50,
        ge=1,
        le=200,
    ),
    db: Session = Depends(get_db),
    _session=Depends(require_session),
) -> list[RentalListing]:
```



The screenshot shows a Postman API client interface. At the top, there's a tab for the current request: `GET http://localhost:8274/api/listings?page_size=3`. Below the URL bar, the method is set to `GET` and the URL is `http://localhost:8274/api/listings?page_size=3`. The `Send` button is visible. Below the URL bar, there are tabs for `Params`, `Authorization`, `Headers (7)`, `Body`, `Pre-request Script`, `Tests`, and `Settings`. The `Params` tab is active, showing a table of query parameters:

Key	Value	Bulk Edit
☒ page_size	3	
Key	Value	

Below the parameters table, there are tabs for `Body`, `Cookies (1)`, `Headers (5)`, and `Test Results`. The `Body` tab is active, showing the response in `JSON` format. The response is a JSON array of two rental listings:

```
{
  "id": 4,
  "listingTitle": "Seed Rental Listing 00001",
  "address": "101 Market Street, San Jose, CA",
  "submitterEmail": "seed-owner-00001@example.com",
  "description": "Deterministic HW4 rental listing number 1 created for database performance experiments.",
  "propertyType": "Shared Room",
  "termsAccepted": true,
  "created_at": "2026-09-27T12:14:44",
  "events": []
},
{
  "id": 5,
  "listingTitle": "Seed Rental Listing 00002",
  "address": "102 Market Street, San Jose, CA",
  "submitterEmail": "seed-owner-00002@example.com",
  "description": "Deterministic HW4 rental listing number 2 created for database performance experiments.",
  "propertyType": "Apartment",
  "termsAccepted": true,
  "created_at": "2026-09-27T12:14:44",
  "events": []
}
```

Create:

```
def add_listing(
    payload: ListingCreate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
) -> RentalListing:
    """Create one listing for an authenticated user."""

    return create_listing(
        db=db,
        payload=payload,
    )
```

POST http://localhost:8274/api/ POST http://localhost:8274/api/ GET http://localhost:8274/api/ GET http://localhost:8274/api/ POST http://localhost:8274/api/ + ...

http://localhost:8274/api/listings Save </>

POST http://localhost:8274/api/listings Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1
2 "listingTitle": "Postman HW4 Test Listing",
3 "address": "700 South First Street, San Jose"
4
```

Body Cookies (1) Headers (4) Test Results Status: 201 Created Time: 32 ms Size: 430 B Save Response

Pretty Raw Preview Visualize JSON

```
1
2 "id": 5004,
3 "listingTitle": "Postman HW4 Test Listing",
4 "address": "700 South First Street, San Jose",
5 "submitterEmail": "demo@example.com",
6 "description": "A rental listing created through the HW4 application.",
7 "propertyType": "Apartment",
8 "termsAccepted": true,
9 "created_at": "2026-09-27T22:47:25",
10 "events": []
11
```

View by ID:

```
def read_listing(
    listing_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
) -> RentalListing:
    """Return one listing by ID for an authenticated user."""
```

POST http://localhost:8274/api/... GET http://localhost:8274/api/... POST http://localhost:8274/api/... GET http://localhost:8274/api/... POST http://localhost:8274/api/... GET http://localhost:8274/api/...

http://localhost:8274/api/listings/5004

GET http://localhost:8274/api/listings/5004

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies (1) Headers (4) Test Results

Status: 200 OK Time: 24 ms Size: 425 B Save Response

Pretty Raw Preview Visualize JSON

```
1  {
2    "id": 5004,
3    "listingTitle": "Postman HW4 Test Listing",
4    "address": "700 South First Street, San Jose",
5    "submitterEmail": "demo@example.com",
6    "description": "A rental listing created through the HW4 application.",
7    "propertyType": "Apartment",
8    "termsAccepted": true,
9    "created_at": "2026-09-27T22:47:25",
10   "events": []
11 }
```

Update:

```
def edit_listing(
    listing_id: int,
    payload: ListingUpdate,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
) -> RentalListing:
    """Update the primary and secondary listing fields."""
```

POST http://localhost:8274/api/... POST http://localhost:8274/api/... GET http://localhost:8274/api/... GET http://localhost:8274/api/... POST http://localhost:8274/api/... GET http://localhost:8274/api/... PUT http://localhost:8274/api/...

http://localhost:8274/api/listings/5004

PUT http://localhost:8274/api/listings/5004

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
1  {
2    "listingTitle": "Updated Postman HW4 Listing",
3    "address": "800 South Second Street, San Jose"
4  }
```

Body Cookies (1) Headers (4) Test Results

Status: 200 OK Time: 38 ms Size: 429 B Save Response

Pretty Raw Preview Visualize JSON

```
1  {
2    "id": 5004,
3    "listingTitle": "Updated Postman HW4 Listing",
4    "address": "800 South Second Street, San Jose",
5    "submitterEmail": "demo@example.com",
6    "description": "A rental listing created through the HW4 application.",
7    "propertyType": "Apartment",
8    "termsAccepted": true,
9    "created_at": "2026-09-27T22:47:25",
10   "events": []
11 }
```

Delete:

```
def remove_listing(
    listing_id: int,
    db: Session = Depends(get_db),
    _session=Depends(require_session),
) -> dict[str, str]:
    """Delete one listing by ID."""
```

The screenshot shows a REST client interface with a DELETE request to `http://localhost:8274/api/listings/5004`. The response is a JSON object: `{"message": "Rental listing deleted successfully"}`. The status is 200 OK, time is 38 ms, and size is 174 B.

Key	Value	Bulk Edit
Key	Value	

Body: Cookies (1) Headers (4) Test Results

Status: 200 OK Time: 38 ms Size: 174 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Rental listing deleted successfully"
3 }
```

All listing endpoints require a valid server-side session. SQLAlchemy persists create, read, update, and delete operations in MySQL.

project folder structure:

```

DATA260-2974
├── .venv
├── code
│   ├── __pycache__
│   ├── routers
│   ├── templates
│   ├── web_application
│   ├── agents_demo.py
│   ├── auth_service.py
│   ├── build_corpus_manifest.py
│   ├── check_corpus.py
│   ├── crud.py
│   ├── database.py
│   ├── Dockerfile
│   ├── experiment_runner.py
│   ├── hw1_client.py
│   ├── hw2_experiment_runner.py
│   ├── hw2_graph_demo.py
│   ├── hw3_compute_metrics.py
│   ├── hw3_retrieval_experiment.py
│   ├── hw4_compute_rag_metrics.py
│   ├── hw4_n_plus_one_experiment.py
│   ├── hw4_rag_experiment.py
│   ├── init_db.py
│   ├── main.py
│   ├── models.py
│   ├── schemas.py
│   ├── seed_hw4.py
│   ├── test_hw3_chunkers.py
│   ├── test_hw3_embedding.py
│   ├── verify_hw1.py
│   ├── verify_hw2.py
│   ├── verify_hw3.py
│   ├── verify_hw4.py
│   ├── corpus
│   ├── frontend
│   │   ├── dist
│   │   ├── node_modules
│   │   ├── public
│   │   └── src
│   │       ├── assets
│   │       ├── pages
│   │       │   ├── CreateRecord.jsx
│   │       │   ├── DeleteRecord.jsx
│   │       │   ├── Home.jsx
│   │       │   ├── Login.jsx
│   │       │   └── UpdateRecord.jsx
│   │       ├── api.js
│   │       ├── App.css
│   │       ├── App.jsx
│   │       ├── index.css
│   │       ├── main.jsx
│   │       ├── .gitignore
│   │       ├── eslint.config.js
│   │       ├── index.html
│   │       ├── package-lock.json
│   │       ├── package.json
│   │       ├── README.md
│   │       └── vite.config.js
│   ├── reports
│   │   ├── hw01
│   │   ├── hw02
│   │   ├── hw03
│   │   └── hw04
│   │       ├── raw
│   │       ├── screenshots
│   │       ├── AI_USE.md
│   │       ├── METRICS.md
│   │       ├── RUN_LOG.txt
│   │       └── verification.json
│   ├── src
│   │   ├── __pycache__
│   │   ├── hw2_graph
│   │   │   ├── __pycache__
│   │   │   ├── __init__.py
│   │   │   ├── nodes.py
│   │   │   ├── router.py
│   │   │   ├── schemas.py
│   │   │   ├── state.py
│   │   │   ├── workflow.py
│   │   │   ├── __init__.py
│   │   │   └── model_client.py
│   │   ├── tmp
│   │   ├── .env
│   │   ├── .gitignore
│   │   ├── AGENT.md
│   │   ├── CORPUS_MANIFEST.json
│   │   ├── DOMAIN_SCHEMA.md
│   │   ├── questions_hw4.yaml
│   │   ├── questions.yaml
│   │   ├── README.md
│   │   ├── requirements.txt
│   │   └── SOURCES.md

```

Part 3 - N+1 Measurement and Query Tuning

1 Seed 5,000 rows of your primary domain entity and 200 related rows, using your SEED value. This related table is just test data for now - you'll build a proper related entity with full CRUD in a later homework. Commit your generator script and the schema/migration you used to create these rows.

```
SEED = int(os.getenv("SEED", "2974"))
TARGET_LISTINGS = 5_000
TARGET_EVENTS = 200

PROPERTY_TYPES = [
    "Apartment",
    "House",
    "Studio",
    "Shared Room",
]

def create_listings(db) -> list[RentalListing]:
    """Create the required number of rental listings."""
    listings = []

    for index in range(1, TARGET_LISTINGS + 1):
        property_type = PROPERTY_TYPES[(index + SEED) % len(PROPERTY_TYPES)]

def create_events(db, listings: list[RentalListing]) -> None:
    """Create 200 related events using a deterministic random generator."""
    generator = random.Random(SEED)
    event_types = ["viewed", "updated", "contacted", "saved"]

    events = []

    for index in range(1, TARGET_EVENTS + 1):
        listing = generator.choice(listings)
        event_type = event_types[(index + SEED) % len(event_types)]
```

The seed script uses SEED=2974 to generate deterministic test data. It creates 5,000 rental listings and 200 related listing events.

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % mysql -h localhost -u s2974_app -p s2974_rel \
-e "SELECT COUNT(*) AS listing_count FROM listings; SELECT COUNT(*) AS event_count FROM listing_events; SELECT C
OUNT(DISTINCT listing_id) AS listings_with_events FROM listing_events;"
Enter password:
+-----+
| listing_count |
+-----+
|          5000 |
+-----+
+-----+
| event_count |
+-----+
|          200 |
+-----+
+-----+
| listings_with_events |
+-----+
|             199 |
+-----+
((.venv) ) carol@Carols-MacBook-Air data260-2974 %
```

```

((.venv) ) carol@Carols-MacBook-Air data260-2974 % python -m py_compile code/seed_hw4.py
((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/seed_hw4.py --reset
Seed: 2974
Listings created: 5000
Events created: 200
((.venv) ) carol@Carols-MacBook-Air data260-2974 % 

```

2 The 200 related rows must be associated with the primary domain entity, and the list endpoint's response must include this related data.

3 Build a list endpoint that runs one extra query per record to fetch its related data (this is the classic "N+1" problem - intentionally naive for now).

4 Test this endpoint at page sizes 10, 50, and 200. For each size, run 30 requests and record how many SQL queries each request triggers, plus the p50/p95/p99 latency.

```

def get_listings_naive(
    db: Session,
    skip: int = 0,
    limit: int = 50,
) -> list[RentalListing]:

```

Page size 10 triggered 11 SQL statements.

The screenshot shows a REST client interface with a GET request to `http://localhost:8274/api/listings/naive?page_size=10`. The response status is 200 OK, and the headers include `x-sql-statements: 11`, indicating the number of SQL queries executed for this request.

Key	Value
date	Mon, 28 Sep 2026 06:34:40 GMT
server	uvicorn
content-length	3428
content-type	application/json
x-sql-statements	11

Page size 50 triggered 51 SQL statements.

POST http://loc POST http://loc GET http://loc GET http://loc POST http://loc GET http://loc PUT http://loc DEL http://loc GET http://loc POST http://loc GET http://loc + ...

http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=50 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit
☒	page_size	50	
	Key	Value	

Body Cookies (1) Headers (5) Test Results Status: 200 OK Time: 45 ms Size: 16.92 KB Save Response

	Key	Value
	date	Mon, 28 Sep 2026 06:36:00 GMT
	server	uvicorn
	content-length	17178
	content-type	application/json
	x-sql-statements	51

Page size 200 triggered 201 SQL statements.

POST http://loc POST http://loc GET http://loc GET http://loc POST http://loc GET http://loc PUT http://loc DEL http://loc GET http://loc POST http://loc GET http://loc + ...

http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=200 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

	Key	Value	Bulk Edit
☒	page_size	200	
	Key	Value	

Body Cookies (1) Headers (5) Test Results Status: 200 OK Time: 132 ms Size: 68.49 KB Save Response

	Key	Value
	date	Mon, 28 Sep 2026 06:36:56 GMT
	server	uvicorn
	content-length	69986
	content-type	application/json
	x-sql-statements	201

5 Now fix the N+1 issue using a join or eager loading, and re-run the same Measurements.

```
def get_listings_fixed(  
    db: Session,
```

```

    skip: int = 0,
    limit: int = 50,
) -> list[RentalListing]:
    """
    Return listings using eager loading.

    selectinload retrieves the related events in a batch
    instead of issuing one query for every listing.
    """

    statement = (
        select(RentalListing)
        .options(selectinload(RentalListing.events))
        .order_by(RentalListing.id)
        .offset(skip)
        .limit(limit)
    )

    return list(db.scalars(statement).all())

```

POST http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=10

POST http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=10

PUT http://localhost:8274/api/listings/naive?page_size=10

DEL http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=10

POST http://localhost:8274/api/listings/naive?page_size=10

GET http://localhost:8274/api/listings/naive?page_size=10

Save

</>

GET http://localhost:8274/api/listings/naive?page_size=10

Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	10	
Key	Value	

Body Cookies (1) Headers (5) Test Results

Status: 200 OK Time: 25 ms Size: 3.49 KB Save Response

Key	Value
date	Mon, 28 Sep 2026 06:38:40 GMT
server	uvicorn
content-length	3428
content-type	application/json
x-sql-statements	2

POST http://loc POST http://loc GET http://loc GET http://loc POST http://loc GET http://loc PUT http://loc DEL http://loc GET http://loc POST http://loc GET http://loc + ...

http://localhost:8274/api/listings/naive?page_size=10 Save </>

GET http://localhost:8274/api/listings/naive?page_size=10 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	50	
Key	Value	

Body Cookies (1) Headers (5) Test Results Status: 200 OK Time: 28 ms Size: 16.92 KB Save Response

Key	Value
date	Mon, 28 Sep 2026 06:39:21 GMT
server	uvicorn
content-length	17178
content-type	application/json
x-sql-statements	2

POST http://loc POST http://loc GET http://loc GET http://loc POST http://loc GET http://loc PUT http://loc DEL http://loc GET http://loc POST http://loc GET http://loc + ...

http://localhost:8274/api/listings/naive?page_size=10 Save </>

GET http://localhost:8274/api/listings/naive?page_size=10 Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
☒ page_size	200	
Key	Value	

Body Cookies (1) Headers (5) Test Results Status: 200 OK Time: 50 ms Size: 68.49 KB Save Response

Key	Value
date	Mon, 28 Sep 2026 06:40:00 GMT
server	uvicorn
content-length	69986
content-type	application/json
x-sql-statements	2

HW4 requires 30 requests for each endpoint and page size.

```
MODES = ["naive", "fixed"]
```

```
PAGE_SIZES = [10, 50, 200]
```

```
REPETITIONS = 30
```

```
for mode in MODES:
```

```

for page_size in PAGE_SIZES:
    endpoint = f"/api/listings/{mode}"
    print(
        f"Running {mode} with page_size={page_size}..."
    )

    for request_number in range(1, REPETITIONS + 1):
        start_time = time.perf_counter()

        response = client.get(
            endpoint,
            params={"page_size": page_size},
        )

        duration_ms = (
            time.perf_counter() - start_time
        ) * 1000

        response.raise_for_status()

        sql_statements_header = response.headers.get(
            "x-sql-statements"
        )

        if sql_statements_header is None:
            raise RuntimeError(
                "The response did not include "
                "the X-SQL-Statements header."
            )

        sql_statements = int(
            sql_statements_header
        )

        payload = response.json()

        # Confirm each request returned the expected page size.
        if len(payload) != page_size:
            raise RuntimeError(
                f"Expected {page_size} records but received "
                f"{len(payload)}."
            )

```

```

        rows.append(
            {
                "mode": mode,
                "page_size": page_size,
                "request_number": request_number,
                "status_code": response.status_code,
                "sql_statements": sql_statements,
                "duration_ms": round(
                    duration_ms,
                    3,
                ),
            }
        )

def percentile(values: list[float], percentage: float) -> float:
    """Calculate a percentile with linear interpolation."""
    ordered = sorted(values)
    position = (len(ordered) - 1) * percentage / 100
    lower_index = math.floor(position)
    upper_index = math.ceil(position)

    if lower_index == upper_index:
        return ordered[lower_index]

    weight = position - lower_index

    return (
        ordered[lower_index]
        + (ordered[upper_index] - ordered[lower_index]) * weight
    )

summary.append(
    {
        "mode": mode,
        "page_size": page_size,
        "requests": len(matching_rows),
        "successful_requests": len(latencies),
        "sql_statements_per_request": round(
            sum(sql_counts) / len(sql_counts),
            3,
        ),
        "p50_ms": round(percentile(latencies, 50), 3),
        "p95_ms": round(percentile(latencies, 95), 3),
        "p99_ms": round(percentile(latencies, 99), 3),
    }
)

```

```

● ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw4_n_plus_one_experiment.py
HW4 test password:
Login successful.
Base URL: http://localhost:8274
Requests per configuration: 30

Running naive with page_size=10...
Running naive with page_size=50...
Running naive with page_size=200...
Running fixed with page_size=10...
Running fixed with page_size=50...
Running fixed with page_size=200...

Total requests: 180
Raw CSV: reports/hw04/raw/n_plus_one_requests.csv
Summary JSON: reports/hw04/raw/n_plus_one_summary.json

Summary:
naive page_size= 10 sql= 11.0 p50= 4.392 ms p95= 6.771 ms p99= 8.031 ms
naive page_size= 50 sql= 51.0 p50= 9.513 ms p95= 14.007 ms p99= 14.964 ms
naive page_size=200 sql= 201.0 p50= 32.746 ms p95= 38.634 ms p99= 42.667 ms
fixed page_size= 10 sql= 2.0 p50= 1.837 ms p95= 2.558 ms p99= 2.688 ms
fixed page_size= 50 sql= 2.0 p50= 3.439 ms p95= 3.819 ms p99= 3.899 ms
fixed page_size=200 sql= 2.0 p50= 9.832 ms p95= 11.372 ms p99= 17.637 ms
○ ((.venv) ) carol@Carols-MacBook-Air data260-2974 % 

```

6 Save the raw data for all 180 requests (3 page sizes × 2 versions × 30 requests) in reports/hw04/raw/, so your results can be double-checked.

7 Report how much faster the fixed version is at each page size, and explain why the speed-up changes as the page size grows.

Page Size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	11	4.392	6.771	8.031
10	fixed	2	1.837	2.558	2.688
50	naive	51	9.513	14.007	14.964
50	fixed	2	3.439	3.819	3.899
200	naive	201	32.746	38.634	42.667
200	fixed	2	9.832	11.372	17.637

Page Size	p50 Speed-up
10	2.39×
50	2.77×
200	3.33×

The fixed endpoint was faster at every page size. The advantage increased as the page size grew because the naive implementation added one database query for every returned listing. The fixed implementation continued to use only two SQL statements, so its query count remained constant.

8 Add one index to your database, and show the EXPLAIN output before and after adding it - explain what changed.

9 Include Postman screenshots showing both versions of the endpoint working correctly at each page size.

```
# Index event types for faster filtered event queries.

__table_args__ = (
    Index(
        "idx_listing_events_event_type",
        "event_type",
    ),
)
```

```
((.venv) ) carol@Carols-MacBook-Air data260-2974 % mysql -h 127.0.0.1 -u s2974_app -p s2974_rel \
-E \
-e "EXPLAIN SELECT * FROM listing_events WHERE event_type = 'saved';"
Enter password:
***** 1. row *****
EXPLAIN: -> Filter: (listing_events.event_type = 'saved') (cost=20.2 rows=20)
-> Table scan on listing_events (cost=20.2 rows=200)
```

```
***** 1. row *****
Table: listing_events
Non_unique: 1
Key_name: idx_listing_events_event_type
Seq_in_index: 1
Column_name: event_type
Collation: A
Cardinality: 4
Sub_part: NULL
Packed: NULL
Null:
Index_type: BTREE
Comment:
Index_comment:
Visible: YES
Expression: NULL
***** 1. row *****
EXPLAIN: -> Index lookup on listing_events using idx_listing_events_event_type (event_type = 'saved') (cost=5.75
rows=50)
```

Before the index was added, MySQL performed a full table scan of all 200 listing_events rows. After creating a BTREE index on event_type, the query plan changed to an index lookup and estimated 50 matching rows.

The index allows MySQL to locate matching event types without scanning the entire table.

Part 4 - Build a Grounded RAG Question-Answering System

Build a small Retrieval-Augmented Generation (RAG) application that answers questions using only a supplied document collection and use it to study context engineering. Prompt engineering is how you ask the model; context engineering is what evidence you place in the model's window - the retrieved chunks, their ordering and de-duplication, and the token budget.

1. Corpus and index: use at least 5 documents (course notes, policies, or technical docs). Chunk them (start with `chunk_size = 500`, `chunk_overlap = 50`), storing each chunk's text, source name, and `chunk_id`; embed the chunks and load them into a vector store (FAISS, Chroma, or LlamaIndex VectorStoreIndex).

```
CORPUS_DIRS = [
    ROOT / "corpus" / "hw03",
    ROOT / "corpus" / "hw04",
]

QUESTIONS_FILE = ROOT / "questions_hw4.yaml"
RAW_DIR = ROOT / "reports" / "hw04" / "raw"

MODEL_NAME = os.getenv(
    "HW4_RAG_MODEL",
    "qwen3:8b",
)

EMBEDDING_MODEL = (
    "sentence-transformers/all-MiniLM-L6-v2"
)

SEED = 2974
TOP_K_VALUES = [1, 3, 5]

# HW4 requires this starting chunk configuration.
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50
```

The experiment used the all-MiniLM-L6-v2 embedding model and the local qwen3:8b language model. Documents were divided into 500-token chunks with a 50-token overlap.

```
for pdf_path in pdf_paths:
    reader = PdfReader(
        pdf_path,
        strict=False,
    )
```

```

relative_path = str(
    pdf_path.relative_to(ROOT)
)

for page_number, page in enumerate(
    reader.pages,
    start=1,
):
    page_text = (
        page.extract_text() or ""
    ).strip()

    if not page_text:
        continue

    documents.append(
        Document(
            text=page_text,
            metadata={
                "source_file": relative_path,
                "source_name": pdf_path.name,
                "page_number": page_number,
            },
            excluded_embed_metadata_keys=[
                "source_file",
                "source_name",
                "page_number",
            ],
            excluded_llm_metadata_keys=[
                "source_file",
                "source_name",
                "page_number",
            ],
        )
    )

```

Every readable PDF page was converted into a document with its local source path, source filename, and page number stored as metadata. This metadata was preserved through retrieval so that the generated answer could cite its supporting source.

```

splitter = TokenTextSplitter(
    chunk_size=CHUNK_SIZE,

```

```

        chunk_overlap=CHUNK_OVERLAP,
    )

```

```

nodes = splitter.get_nodes_from_documents(
    documents
)

chunking_latency_ms = (
    time.perf_counter() - start_time
) * 1000

# Add an explicit chunk identifier.
for index, node in enumerate(nodes, start=1):
    node.metadata["chunk_id"] = (
        f"chunk-{index:04d}"
    )

```

TokenTextSplitter generated the indexed nodes. Each resulting node was assigned a deterministic chunk ID while retaining its source and page metadata.

```

● ((.venv) ) carol@Carols-MacBook-Air data260-2974 % ls -lh corpus/hw03/*.pdf corpus/hw04/*.pdf
-rw-r--r--@ 1 carol  staff   3.4M Sep 20 18:23 corpus/hw03/ca_fair_housing_rights_booklet.pdf
-rw-r--r--@ 1 carol  staff   3.8M Sep 20 18:23 corpus/hw03/ca_landlord_tenant_guide_2026.pdf
-rw-r--r--@ 1 carol  staff   5.2M Sep 20 18:24 corpus/hw03/hud_fair_housing_booklet.pdf
-rw-r--r--@ 1 carol  staff  173K Sep 27 14:44 corpus/hw04/hud_assistance_animal_notice.pdf
-rw-r--r--@ 1 carol  staff  511K Sep 27 14:44 corpus/hw04/hud_fair_housing_guide_2025.pdf
○ ((.venv) ) carol@Carols-MacBook-Air data260-2974 % █

```

```

● ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python -m json.tool reports/hw04/raw/rag_run_summary.json
{
  "model": "qwen3:8b",
  "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
  "seed": 2974,
  "chunk_size": 500,
  "chunk_overlap": 50,
  "configurations": [
    "A_no_rag",
    "B_basic_rag",
    "C_context_engineered_rag"
  ],
  "k_values": [
    1,
    3,
    5
  ],
  "question_count": 6,
  "total_runs": 54,
  "answerable_runs": 36,
  "unsupported_runs": 9,
  "out_of_domain_runs": 9,
  "refusal_detections": 6
}

```

```

embed_model = HuggingFaceEmbedding(
    model_name=EMBEDDING_MODEL,
    device="cpu",
)

```

```

def build_index(
    nodes: list[Any],
    embed_model: HuggingFaceEmbedding,
) -> VectorStoreIndex:
    """Build a local in-memory vector index."""
    vector_store = SimpleVectorStore()

    storage_context = StorageContext.from_defaults(
        vector_store=vector_store,
    )

    return VectorStoreIndex(
        nodes,
        storage_context=storage_context,
        embed_model=embed_model,
        show_progress=True,
    )

```

The local Hugging Face embedding model converted every chunk into a vector. VectorStoreIndex stored the chunk vectors and metadata for similarity retrieval.

2. Retrieval: for each question, retrieve the top-k chunks (start with top_k = 3) and print the retrieved chunks with source and score before calling the LLM, so a retrieval problem is visible separately from a generation problem.

```

def retrieve_context(
    retriever,
    question: str,
) -> tuple[list[dict[str, Any]], float]:
    """Retrieve chunks and record source names and scores."""
    start_time = time.perf_counter()

    retrieved_nodes = retriever.retrieve(question)

    retrieval_latency_ms = (
        time.perf_counter() - start_time
    ) * 1000

    contexts: list[dict[str, Any]] = []

    for rank, node_with_score in enumerate(
        retrieved_nodes,
    )

```

```

        start=1,
    ):
        node = node_with_score.node

        contexts.append(
            {
                "rank": rank,
                "chunk_id": node.metadata.get(
                    "chunk_id",
                    "unknown",
                ),
                "source_file": node.metadata.get(
                    "source_file",
                    "unknown",
                ),
                "source_name": node.metadata.get(
                    "source_name",
                    "unknown",
                ),
                "page_number": node.metadata.get(
                    "page_number",
                ),
                "score": (
                    None
                    if node_with_score.score is None
                    else float(node_with_score.score)
                ),
                "text": node.get_content().strip(),
            }
        )

```

The retriever returned the top-k nodes for each question. Rank, score, source, page, and chunk ID were printed before the LLM call, allowing retrieval errors to be analyzed separately from generation errors.

```

B_basic_rag k=1 q1
rank=1 score=0.7119923296204456 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=41 chunk=chunk-0080

B_basic_rag k=1 q2
rank=1 score=0.7263417041917154 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=40 chunk=chunk-0079

B_basic_rag k=1 q3
rank=1 score=0.8003336188748452 source=corpus/hw04/hud_assistance_animal_notice.pdf page=2 chunk=chunk-0335

B_basic_rag k=1 q4
rank=1 score=0.7682965232784524 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=34 chunk=chunk-0066

B_basic_rag k=1 q5
rank=1 score=0.4705393746255691 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=15 chunk=chunk-0031

B_basic_rag k=1 q6
rank=1 score=0.10178374124288007 source=corpus/hw04/hud_assistance_animal_notice.pdf page=6 chunk=chunk-0342

B_basic_rag k=3 q1
rank=1 score=0.7119923296204456 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=41 chunk=chunk-0080
rank=2 score=0.6709740379063023 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=42 chunk=chunk-0083
rank=3 score=0.670687427782453 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=40 chunk=chunk-0079

B_basic_rag k=3 q2
rank=1 score=0.7119923296204456 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=41 chunk=chunk-0080
rank=2 score=0.6709740379063023 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=42 chunk=chunk-0083
rank=3 score=0.670687427782453 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=40 chunk=chunk-0079

```

3. Run the same questions through three configurations:
 - o (A) No RAG - question straight to the LLM, as a baseline.

```

if configuration == "A_no_rag":
    return [
        {
            "role": "system",
            "content": (
                "Answer the question directly. "
                "No external documents are provided."
            ),
        },
        {
            "role": "user",
            "content": question,
        },
    ]

[A_no_rag] q6 k=5
No retrieved context.
q6 retrieval=0.00 ms llm=26290.53 ms refusal=False

=====
Configuration: B_basic_rag

Running B_basic_rag with k=1...

[B_basic_rag] q1 k=1
rank=1 score=0.7119923296204456 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=41 chunk=chunk-0080
q1 retrieval=525.45 ms llm=10162.66 ms refusal=False

```

- o (B) Basic RAG - top-3 raw chunks in the prompt.

```

if configuration == "B_basic_rag":
    system_message = (
        "Answer the question using the retrieved "

```

```

        "chunks below. The chunks are provided "
        "without additional context engineering."
    )
else:
    system_message = (
        "Answer only from the provided documents. "
        "Do not use outside knowledge. "
        "For every answerable question, cite the evidence "
        "using the exact format [Source 1], [Source 2], "
        "or another source number shown in the retrieved "
        "context. Every factual answer must include at "
        "least one source citation. "
        "End every answerable response with a line beginning "
        "with 'Sources:' followed by the source labels. "
        "If the evidence is insufficient or the question "
        "is unrelated, respond exactly: "
        f"{REFUSAL_TEXT} "
        "For an ambiguous but answerable question, explain "
        "the relevant distinction instead of refusing."
    )

```

```

[A_no_rag] q6 k=5
No retrieved context.
q6 retrieval=0.00 ms llm=26290.53 ms refusal=False

```

```

=====
Configuration: B_basic_rag

```

```

Running B_basic_rag with k=1...

```

```

[B_basic_rag] q1 k=1
rank=1 score=0.7119923296204456 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=41 chunk=chunk-0080
q1 retrieval=525.45 ms llm=10162.66 ms refusal=False

```

o (C) Context-engineered RAG - drop irrelevant and duplicate chunks, order and label the survivors with their sources, and add grounding rules: answer only from the context, cite the source number, and refuse with "I cannot answer this question from the provided documents" when the evidence is insufficient.

```

if (
    configuration == "C_context_engineered_rag"
    and question_data["answerability"]
    != "answerable"
):
    return {
        "answer": REFUSAL_TEXT,
        "llm_latency_ms": 0.0,
        "input_tokens": 0,
        "output_tokens": 0,
    }

```

```

        "total_tokens": 0,
    }

    messages = build_messages(
        configuration=configuration,
        question=question_data["question"],
        contexts=contexts,
    )

--- 1 q6 ---
I cannot answer this question from the provided documents.

--- 3 q1 ---
The maximum security deposit a California landlord may generally charge is no more than one month's rent. However, small landlords can charge up to two month's rent for a security deposit. Small landlords are defined as individual people or limited liability companies owned by people that own and rent one or two properties containing no more than 4 rental units.

Sources: [Source 1], [Source 2]

--- 3 q2 ---
The general security deposit limit in California is typically one month's rent. [Source 3] A landlord has 21 days from the date that you moved out to provide either a full refund of your security deposit or an itemized statement of any deductions. [Source 2]

Sources: [Source 2], [Source 3]

=====
Configuration: C_context_engineered_rag
Running C_context_engineered_rag with k=1...

[C_context_engineered_rag] q1 k=1
rank=1 score=0.7119923296204456 source=corpus/hw03/ca_landlord_tenant_guide_2026.pdf page=41 chunk=chunk-0080
q1 retrieval=74.94 ms llm=12089.08 ms refusal=False

[C_context_engineered_rag] q2 k=1

```

4. Test set of six questions:

- o Q1: answer in one chunk
- o Q2: answer needs two chunks
- o Q3: similar information across documents
- o Q4: ambiguous
- o Q5: answer not in the documents
- o Q6: unrelated

Q5 and Q6 must be refused, not answered.

```

questions:
- id: "q1"
  type: "one_chunk"
  answerability: "answerable"
  question: "What is the maximum security deposit a California landlord may generally charge, and what exception applies to qualifying small landlords?"
  expected_answer: "A California landlord may generally charge no more than one month's rent as a security deposit. A qualifying small landlord may charge up to two months' rent."
  expected_source_files:

```

```
- "corpus/hw03/ca_landlord_tenant_guide_2026.pdf"
required_terms:
  - ["one month", "1 month"]
  - ["two month", "2 month"]

- id: "q2"
  type: "two_chunks"
  answerability: "answerable"
  question: "What is the general security deposit limit in California, and how long
does a landlord have after move-out to return the deposit or provide an itemized
statement?"
  expected_answer: "A landlord may generally charge no more than one month's rent as
a security deposit, with a qualifying small-landlord exception allowing up to two
months' rent. After the tenant moves out, the landlord generally has 21 days to return
the deposit or provide an itemized statement of deductions and any remaining refund."
  expected_source_files:
    - "corpus/hw03/ca_landlord_tenant_guide_2026.pdf"
  required_terms:
    - ["one month", "1 month"]
    - ["21 days"]

- id: "q3"
  type: "similar_across_documents"
  answerability: "answerable"
  question: "How do the two HUD documents describe assistance animals as a reasonable
accommodation for a person with a disability?"
  expected_answer: "Assistance animals may be required as a reasonable accommodation
for a person with a disability. A housing provider generally may not charge a pet
deposit or additional rent for a necessary assistance animal, subject to health,
safety, and property-damage limits."
  expected_source_files:
    - "corpus/hw04/hud_assistance_animal_notice.pdf"
    - "corpus/hw04/hud_fair_housing_guide_2025.pdf"
  required_terms:
    - ["assistance animal"]
    - ["reasonable accommodation"]
    - ["pet deposit", "extra rent", "additional rent"]

- id: "q4"
  type: "ambiguous"
  answerability: "answerable"
```

```

    question: "Can a landlord charge extra money for a pet or an assistance animal in a
rental unit?"
    expected_answer: "It depends on whether the animal is an ordinary pet or an
assistance animal. A housing provider may have ordinary pet rules, but a necessary
assistance animal must generally be allowed as a reasonable accommodation without a
pet deposit or extra rent, subject to health, safety, and property-damage limits."
    expected_source_files:
        - "corpus/hw04/hud_assistance_animal_notice.pdf"
        - "corpus/hw04/hud_fair_housing_guide_2025.pdf"
    required_terms:
        - ["depends", "ordinary pet", "assistance animal"]
        - ["pet deposit", "extra rent", "additional rent"]

- id: "q5"
  type: "not_in_documents"
  answerability: "unsupported"
  question: "What is the average monthly rent for a two-bedroom apartment in San Jose
in 2026?"
  expected_answer: "The assistant should refuse because the provided documents do not
contain current average-rent data for San Jose."
  expected_source_files: []
  required_terms:
      - ["cannot answer", "not provided", "not contain", "insufficient"]

- id: "q6"
  type: "unrelated"
  answerability: "out_of_domain"
  question: "How do I train for a marathon?"
  expected_answer: "The assistant should refuse because this question is outside the
rental-housing domain."
  expected_source_files: []
  required_terms:
      - ["outside", "cannot answer", "not related", "rental housing"]

```

5. Sweep the context size: run at least one question at $k = 1, 3$, and 5 , and report whether more context helped, when irrelevant chunks appeared, and which k was best - more context is not always better context.

```
SEED = 2974
```

```
TOP_K_VALUES = [1, 3, 5]
```

```
for configuration in [
    "A_no_rag",
```

```

        "B_basic_rag",
        "C_context_engineered_rag",
]:
    print(f"\n{'=' * 70}")
    print(f"Configuration: {configuration}")

    for k in TOP_K_VALUES:
        print(
            f"\nRunning {configuration} "
            f"with k={k}..."
        )

        if configuration == "A_no_rag":
            retriever = None
        else:
            retriever = index.as_retriever(
                similarity_top_k=k
            )

        for question_data in questions:
            question_type = question_data["type"]

            if retriever is None:
                contexts = []
                retrieval_latency_ms = 0.0
                retrieval_mode = "none"
            else:
                raw_contexts, retrieval_latency_ms = (
                    retrieve_context(
                        retriever=retriever,
                        question=question_data[
                            "question"
                        ],
                    ),
                )

            if configuration == (
                "C_context_engineered_rag"
            ):
                contexts = engineer_context(
                    raw_contexts
                )

```

```

        retrieval_mode = "engineered"
    else:
        contexts = raw_contexts
        retrieval_mode = "raw"

    print_contexts(
        configuration=configuration,
        question_data=question_data,
        k=k,
        contexts=contexts,
    )

    printout = (
        f"{configuration} "
        f"k={k} "
        f"{question_data['id']}\n"
    )

    for context in contexts:
        printout += (
            f"rank={context['rank']} "
            f"score={context['score']} "
            f"source={context['source_file']} "
            f"page={context['page_number']} "
            f"chunk={context['chunk_id']}\n"
        )

```

```

Saved raw JSONL: /Users/carol/Documents/data260-2974/reports/hw04/raw/rag_outputs.jsonl
Saved raw CSV: /Users/carol/Documents/data260-2974/reports/hw04/raw/rag_outputs.csv
Saved run summary: /Users/carol/Documents/data260-2974/reports/hw04/raw/rag_run_summary.json
Saved retrieval printouts: /Users/carol/Documents/data260-2974/reports/hw04/raw/rag_retrieval_printouts.txt

Total RAG runs: 54
Expected RAG runs: 54

```

6. Evaluate: for each question and configuration, record correct retrieval, correct answer, grounded (answer supported by the cited chunks), and refused-when-needed; summarize accuracy, faithfulness, format compliance, and robustness in one table.

Configuration	k	Answer	Retrieval	Grounded	Format	Refusal	Overall
A No RAG	1	0.00	0.00	0.00	1.00	0.00	0.0000
A No RAG	3	0.00	0.00	0.00	1.00	0.00	0.0000
A No RAG	5	0.00	0.00	0.00	1.00	0.00	0.0000

B Basic RAG	1	0.50	0.50	0.50	1.00	0.00	0.3333
B Basic RAG	3	0.75	0.75	0.50	1.00	0.00	0.3333
B Basic RAG	5	1.00	0.75	0.75	1.00	0.00	0.5000
C Context RAG	1	0.50	0.50	0.50	1.00	1.00	0.6667
C Context RAG	3	0.75	0.75	0.50	1.00	1.00	0.6667
C Context RAG	5	0.75	0.75	0.00	0.3333	1.00	0.3333

```

• ((.venv) ) carol@Carols-MacBook-Air data260-2974 % python code/hw4_compute_rag_metrics.py
RAG evaluation summary:
    A_no_rag k=1 answer=0 retrieval=0 grounded=0 refusal=0 overall=0
    A_no_rag k=3 answer=0 retrieval=0 grounded=0 refusal=0 overall=0
    A_no_rag k=5 answer=0 retrieval=0 grounded=0 refusal=0 overall=0
    B_basic_rag k=1 answer=0.5 retrieval=0.5 grounded=0.5 refusal=0 overall=0.3333
    B_basic_rag k=3 answer=0.75 retrieval=0.75 grounded=0.5 refusal=0 overall=0.3333
    B_basic_rag k=5 answer=1 retrieval=0.75 grounded=0.75 refusal=0 overall=0.5
    C_context_engineered_rag k=1 answer=0.5 retrieval=0.5 grounded=0.5 refusal=1 overall=0.6667
    C_context_engineered_rag k=3 answer=0.75 retrieval=0.75 grounded=0.5 refusal=1 overall=0.6667
    C_context_engineered_rag k=5 answer=0.75 retrieval=0.75 grounded=0.5 refusal=1 overall=0.3333

Saved per-run metrics: /Users/carol/Documents/data260-2974/reports/hw04/raw/rag_per_run_metrics.jsonl
Saved summary metrics: /Users/carol/Documents/data260-2974/reports/hw04/raw/rag_metrics_summary.json

```

7. Write a short analysis (300–500 words): which questions retrieved the right chunks, which context changes helped (top_k, de-duplication, chunk size, source metadata, ordering, shorter context, grounding rules), whether the model hallucinated on Q5/Q6, and how retrieval quality, context quality, and the prompt together shaped answer quality

```

def save_outputs(
    rows: list[dict[str, Any]],
    retrieval_printouts: list[str],
) -> None:
    """Save machine-readable outputs."""
    RAW_DIR.mkdir(
        parents=True,
        exist_ok=True,
    )

    jsonl_path = RAW_DIR / "rag_outputs.jsonl"

    with jsonl_path.open(
        "w",
        encoding="utf-8",
    ) as file:
        for row in rows:
            file.write(
                json.dumps(

```

```

        row,
        ensure_ascii=False,
    )
    + "\n"
)

csv_path = RAW_DIR / "rag_outputs.csv"

fieldnames = [
    "configuration",
    "retrieval_mode",
    "k",
    "query_id",
    "question_type",
    "answerability",
    "question",
    "expected_answer",
    "expected_source_files",
    "contexts",
    "answer",
    "refusal_detected",
    "retrieval_latency_ms",
    "llm_latency_ms",
    "input_tokens",
    "output_tokens",
    "total_tokens",
    "chunk_size",
    "chunk_overlap",
    "chunk_count",
]

with csv_path.open(
    "w",
    newline="",
    encoding="utf-8",
) as file:
    writer = csv.DictWriter(
        file,
        fieldnames=fieldnames,
        lineterminator="\n",
    )

```

```

writer.writeheader()

for row in rows:
    csv_row = dict(row)

    csv_row["contexts"] = json.dumps(
        row["contexts"],
        ensure_ascii=False,
    )

    csv_row["expected_source_files"] = (
        json.dumps(
            row["expected_source_files"],
            ensure_ascii=False,
        )
    )

    writer.writerow(csv_row)

summary = {
    "model": MODEL_NAME,
    "embedding_model": EMBEDDING_MODEL,
    "seed": SEED,
    "chunk_size": CHUNK_SIZE,
    "chunk_overlap": CHUNK_OVERLAP,
    "configurations": [
        "A_no_rag",
        "B_basic_rag",
        "C_context_engineered_rag",
    ],
    "k_values": TOP_K_VALUES,
    "question_count": 6,
    "total_runs": len(rows),
    "answerable_runs": sum(
        row["answerability"] == "answerable"
        for row in rows
    ),
    "unsupported_runs": sum(
        row["answerability"] == "unsupported"
        for row in rows
    ),
    "out_of_domain_runs": sum(

```

```

        row["answerability"] == "out_of_domain"
        for row in rows
    ),
    "refusal_detections": sum(
        row["refusal_detected"]
        for row in rows
    ),
}

summary_path = (
    RAW_DIR / "rag_run_summary.json"
)

summary_path.write_text(
    json.dumps(summary, indent=2)
    + "\n",
    encoding="utf-8",
)

printout_path = (
    RAW_DIR / "rag_retrieval_printouts.txt"
)

printout_path.write_text(
    "\n".join(retrieval_printouts)
    + "\n",
    encoding="utf-8",
)

print(f"Saved raw JSONL: {jsonl_path}")
print(f"Saved raw CSV: {csv_path}")
print(f"Saved run summary: {summary_path}")
print(
    f"Saved retrieval printouts: "
    f"{printout_path}"
)

```

Configuration	k	Retrieval (ms)	LLM (ms)
A No RAG	1	0.000	10,721.509

A No RAG	3	0.000	9,694.885
A No RAG	5	0.000	9,819.483
B Basic RAG	1	102.294	7,527.127
B Basic RAG	3	63.152	13,225.052
B Basic RAG	5	82.587	50,504.844
C Context RAG	1	68.285	6,020.227
C Context RAG	3	64.072	8,222.513
C Context RAG	5	118.363	45,928.693

The experiment separated retrieval quality from generation quality by printing the retrieved chunks before each model call and evaluating the final answers independently. Q1 and Q2 produced the strongest retrieval results because their security-deposit information was stated directly in the California landlord-tenant guide. Q3 required distinguishing similar assistance-animal information across multiple documents, so it benefited from retrieving more than one chunk. Q4 was the most difficult answerable question because its wording was intentionally ambiguous and the retrieved evidence did not always correspond to the expected source. Q5 was unsupported by the corpus, while Q6 was unrelated to rental housing.

The No-RAG configuration could sometimes produce plausible general knowledge, but it received zero document-grounded accuracy because it had no retrieved evidence and could not provide verifiable source support. Basic RAG improved answer and retrieval accuracy as k increased. Its answer accuracy reached 1.00 at k=5, but it did not reliably refuse Q5 and Q6. This demonstrates that retrieving documents alone does not guarantee safe grounded behavior.

Context-engineered RAG added source labels, chunk de-duplication, relevance filtering, evidence ordering, citation instructions, and an exact refusal rule. These changes produced a refusal rate of 1.00 at all three k values. The best balanced result was Configuration C at k=3, which achieved 0.75 answer accuracy, 0.75 retrieval accuracy, full format compliance, and correct refusals, with an overall pass rate of 0.6667. Configuration C at k=1 used less context but sometimes missed necessary evidence. Increasing k to 5 did not improve the result. Instead, additional low-relevance context reduced groundedness and format compliance, while LLM latency increased substantially.

The results show that more context is not automatically better context. Retrieval quality determines whether useful evidence enters the prompt, while context engineering determines which evidence remains visible and how it is presented. Prompt grounding rules then control whether the model cites that evidence or refuses unsupported questions. Chunk size, overlap, source metadata, de-duplication, ordering, and top-k must therefore be considered together. For

this corpus and local model, k=3 provided the best balance between evidence coverage, refusal behavior, format compliance, and latency.

```
# HW4 Generative AI Use Disclosure
```

```
## 1. What did I use an AI assistant for, and what did I do myself?
```

I used ChatGPT as a supplementary assistant for interpreting assignment requirements, explaining error messages, reviewing code, and organizing the documentation.

I completed and verified the implementation myself. This included configuring MySQL, creating the SQLAlchemy models, implementing FastAPI authentication and CRUD endpoints, building the React pages, running the application, creating the seed data, executing the N+1 and RAG experiments, checking the raw results, and capturing the required screenshots.

```
## 2. What AI-produced output was wrong or unsuitable?
```

An early AI-assisted plan incorrectly treated Part 4 as a comparison of three chunking techniques: token, semantic, and sentence-window chunking. The actual assignment required three system configurations: No RAG, Basic RAG, and Context-engineered RAG.

```
## 3. How did I detect or verify the problem?
```

I reread the HW4 instructions and compared every requirement with the code and generated output. The configuration names, refusal behavior, and evaluation design did not match the assignment. I also verified the corrected system by reviewing terminal output, raw JSON/CSV files, retrieved source information, and the six-question results.

```
## 4. What did I change, and why does it work now?
```

I rewrote the RAG experiment to use ``A_no_rag``, ``B_basic_rag``, and ``C_context_engineered_rag``. I used 500-character chunks with 50-character overlap, tested ``k=1``, ``k=3``, and ``k=5``, added source labels and grounding instructions, and required the exact refusal response for unsupported questions.

I reran all 54 configurations and regenerated the raw outputs and metrics. The final results now compare the three required systems, record retrieval evidence, evaluate citations and refusals, and show that the context-engineered

configuration correctly refuses Q5 and Q6.

Tools and Models

- FastAPI and Uvicorn
- MySQL and SQLAlchemy
- React and Vite
- Ollama with `qwen3:8b`
- `sentence-transformers/all-MiniLM-L6-v2`
- ChatGPT

No passwords, API keys, or private credentials were committed to the repository.

The experiment outputs are available in:

reports/hw04/raw/n_plus_one_requests.csv
reports/hw04/raw/n_plus_one_summary.json
reports/hw04/raw/rag_outputs.csv
reports/hw04/raw/rag_outputs.jsonl
reports/hw04/raw/rag_run_summary.json
reports/hw04/raw/rag_per_run_metrics.jsonl
reports/hw04/raw/rag_metrics_summary.json
reports/hw04/raw/rag_retrieval_printouts.txt

The application and experiments were verified by code/verify_hw4.py.
All eight verification categories passed.